

KĨ NĂNG VIBECODING

LÀM GÌ KHI CÓ UNEXPECTED ERRORS TRONG QUÁ TRÌNH TRIỂN KHAI CODE THEO SDD

Hãy đi theo 4 bước:

1. Dừng lại và phân loại vấn đề
 - Đây là bug triển khai?
 - Thiếu sót của spec?
 - Business requirement đổi?
 - Hay chỉ là edge case chưa được tính?
2. Ghi nhận thành thay đổi chính thức
 - Không sửa “ngầm”
 - Cập nhật spec hoặc tạo một spec delta
 - Nêu rõ lý do thay đổi
3. Đánh giá ảnh hưởng
 - Ảnh hưởng tới data model?
 - Ảnh hưởng tới API?
 - Ảnh hưởng tới test?
 - Ảnh hưởng tới flow business?
4. Chỉ rồi mới triển khai tiếp
 - Sau khi spec được cập nhật và đồng ý, mới code tiếp
 - Nếu cần, rollback phần đang làm dở

Phân biệt 3 loại thay đổi

A. Bug kỹ thuật

Ví dụ:

- timeout
- deadlock
- memory leak
- lỗi concurrency

Cách xử lý:

- sửa code
- thêm test

- nếu cần thì cập nhật spec phần constraint kỹ thuật

B. Spec thiếu

Ví dụ:

- spec chưa nói rõ trường hợp tài liệu bị lỗi format
- chưa định nghĩa trạng thái failure

Cách xử lý:

- cập nhật spec
- thêm acceptance criteria
- rồi mới implement tiếp

C. Business logic đổi

Ví dụ:

- khách hàng muốn đổi thứ tự xử lý
- rule phân loại tài liệu thay đổi
- output cần thêm/bớt trường dữ liệu

Cách xử lý:

- coi như change request chính thức
- không tự ý “suy diễn”
- cần confirm lại requirement trước khi sửa

Bạn là AI agent đang làm việc theo spec-driven development.

Nhiệm vụ của bạn:

- Phân tích lỗi hoặc thay đổi yêu cầu hiện tại.
- Xác định xem đây là:
 - 1) bug triển khai,
 - 2) thiếu sót trong spec,
 - 3) thay đổi business logic,
 - 4) hay edge case chưa được mô tả.
- Nếu lỗi làm spec hiện tại không còn đúng hoặc không đủ rõ, hãy cập nhật spec trước khi tiếp tục code.

Yêu cầu khi cập nhật spec:

1. Không sửa code ngay nếu spec chưa được làm rõ.
2. Ghi rõ:
 - vấn đề phát sinh là gì
 - nguyên nhân hoặc giả định nguyên nhân
 - phần spec nào bị ảnh hưởng

- spec mới cần thay đổi như thế nào
- 3. Nếu có nhiều cách giải quyết, đề xuất phương án tốt nhất và nêu lý do.
- 4. Nếu thay đổi ảnh hưởng tới:
 - API contract
 - database schema
 - business rules
 - acceptance criteriathì phải nêu rõ impact.
- 5. Sau khi cập nhật spec, hãy liệt kê các bước tiếp theo:
 - test nào cần cập nhật
 - code nào cần sửa
 - rủi ro nào cần theo dõi

Đầu ra mong muốn:

- Một bản spec đã được cập nhật rõ ràng
- Danh sách thay đổi so với spec cũ
- Các việc cần làm tiếp theo để triển khai an toàn

Nếu thông tin chưa đủ rõ, hãy dừng lại và hỏi lại thay vì tự suy diễn.

PROMPT HIỂU TASK TRƯỚC KHI BẮT ĐẦU

Đóng vai trò là một Senior Software Engineer/AI Developer. Tôi đang làm việc theo phương pháp Spec-Driven Development (sử dụng OpenSpec + Beads) và chuẩn bị thực thi một task mới.

Mục tiêu: hiểu thật tường minh về bead này, đảm bảo logic hệ thống và khả năng bảo trì trước khi viết bất kỳ dòng code nào.

- ****Bead ID & Title:**** [ví dụ: beads-042 - Implement conversation memory persistence]
 - ****Nội dung Bead:**** [Paste output của `bd show <id>` vào đây]
 - ****OpenSpec artifact liên quan:**** [Paste hoặc link tới design.md / specs.md của change]
 - ****Stack/Context:**** Python/FastAPI/LangGraph, PostgreSQL, Qdrant hybrid search
 - ****ADRs liên quan (nếu biết):**** [ví dụ: ADR-006 - Backend RAG contract]
-

Phân tích và trả về báo cáo ngắn gọn:

1. ****Core Objective:**** 1-2 câu - bead này đảm nhận vai trò gì và tại sao cần thiết.

2. ****Data Flow:****
 - Inputs: loại dữ liệu, schema, source
 - Outputs: format mong đợi, destination (DB? API response? queue?)
3. ****Dependencies & Integration Points:****
 - Phụ thuộc vào module/bead nào?
 - ADR nào chi phối hành vi của bead này?
 - Tác động đến phần nào của hệ thống nếu implementation sai?
4. ****Constraints & Edge Cases:**** Rủi ro kỹ thuật, giới hạn, các trường hợp ngoại lệ cần handle.
5. ****Actionable Steps:**** 3-5 bước kỹ thuật nhỏ (pseudo-code hoặc logic flow) để bắt đầu code ngay.
6. ****Definition of Done:**** Điều kiện cụ thể để bead này được coi là hoàn thành (integration test pass, API response đúng schema, v.v.).
7. ****Verification Approach:**** Cách verify nhanh nhất mà không cần chạy full test suite – ví dụ: curl command, unit test case, log output cần thấy.

REVIEW CODE VỚI BUGBOT

You are reviewing this implementation as a Senior Software Engineer.

Review ONLY the code changed in this task.

Compare the implementation against the OpenSpec documentation.

Look for:

1. Missing requirements from the spec
2. Incorrect implementation
3. Edge cases not handled
4. Possible regressions
5. Logic bugs
6. State management issues
7. Race conditions
8. Error handling
9. Performance problems
10. Maintainability issues

For every finding:

- explain why it is a problem

- reference the related requirement in OpenSpec
- estimate severity (Critical / High / Medium / Low)
- propose the smallest possible fix

If everything matches the spec, explicitly state that no issue was found.

PROMPT FIX LỖI SAU KHI AI AGENT REVIEW CODE ĐƯỢC SINH RA

Read the problems

You are implementing fixes based on a completed code review.

Your task is to fix ONLY the issues identified in the review.

Requirements:

- Do not introduce new features.
- Do not refactor unrelated code.
- Preserve the existing architecture.
- Keep the changes as small as possible.
- Follow the OpenSpec documentation exactly.
- Preserve backward compatibility unless the review explicitly requires otherwise.

For each issue:

1. Explain the root cause.
2. Describe the minimal change needed.
3. Implement the fix.
4. Verify that the fix does not affect unrelated functionality.

When finished, summarize:

- Issues fixed
- Files changed
- Remaining risks (if any)

PROMPT TÓM TẮT CHỨC NĂNG

Act as my mentor.

I have just finished implementing a feature/function.

Without focusing on low-level implementation, explain:

1. What business problem does this feature solve?
2. What triggers this feature?
3. Who are the actors?
4. What is the expected output?
5. What are the success and failure scenarios?

If my implementation has assumptions or hidden business rules, point them out.

Answer in Vietnamese.